

2143 Object-Oriented Programming

Comprehensive Study Guide — Spring 2026

Answer each question in your own words or in code as appropriate.

Questions marked (*Trace*) require exact output. Questions marked (*Code*) require written C++ code.

Inheritance

- Q1.** (*Code*) Assume there is a class called Pixel and it has three data members: Where: I would like you to add a method called GrayScale that turns a pixel into a shade of gray. To do this you can simply average the three color values together, then replace the Red, Green, and Blue data members with the average of all three color channels, so that each color channel has the same value.

```
1 int Red;    //possible values are 0-255
2 int Green;  //possible values are 0-255
3 int Blue;   //possible values are 0-255
```

- Q2.** Which of the protection mechanisms do we use so that a sub class (a class that inherits from another class) can gain access to sensitive portions of the class?
- Q3.** Determine which to use: Inheritance or Composition Laptop [IS-A] Computer Smartphone [HAS-A] Touchscreen Compiler [IS-A] Program Car [HAS-A] GPS System Python [IS-A] Programming Language Web Browser [HAS-A] Address Bar User Interface [HAS-A] Button A CPU [IS-A] Hardware Component Computer [HAS-A] CPU A LinkedList [IS-A] Data Structure A Dog [IS-A] Animal A Car [HAS-A] Engine A Car [IS-A] Vehicle A Square [IS-A] Shape A Line [HAS-A] Point Inheritance Types Label each Ascii art drawing with its appropriate inheritance type. [Single] [Multi-Level] [Hierarchical] [Multiple] [Hybrid]

- Q4.** Deriving a class in such a way that the base class members are not available for further inheritance is known as
- Q5.** Which access specifiers should be used so that all the derived classes restrict further inheritance of base class members?
- Q6.** Which type of inheritance is most suitable for inheriting Same syllabus into different colleges with different streams?
- Q7.** Which problem arises due to multiple inheritance, if hierarchical inheritance is used previously for its base classes?
- Q8.** If multi-level inheritance is used, First class B inherits class A, then C inherits B and so on. Till how many classes can this go on?
- Q9.** Inheritance - “Is-a” relationships (e.g., **Car** is a **Vehicle**). - Base (parent) and derived (child) classes.
- Q10.** Is there ever a time you would use private in a base class and prevent access to a derived class? Give an example.
- Q11.** Give a concise example in C++ showing: - Simple Inheritance - Multi-Level Inheritance - Multiple Inheritance - Hierarchical Inheritance
- Q12.** By defining a base abstract method as public pure virtual, are you guaranteeing that protected or private access in derived classes won't cause conflicts? (Trick question, reflect on access levels!)
- Q13.** Which phrase describes an inheritance (“is-a”) relationship?

Abstract Classes

- Q14.** (*Code*) Write a class definition called `RgbColor` to represent an RGB color. Even if we haven't discussed what an RGB color is, it is pretty intuitive. It contains 3 values, 1 for each of the Red, Green, and Blue, color channels. A valid value for each channel is between 0-255 inclusive. Examples: Methods: Usage: Look at the following to help you determine what will be needed as part of your class: Answer On Next Page...

```

1 RgbColor Color1;                                     // each color = 0
2 RgbColor Color2(0.30,1.0,0.50);   // each color a percentage of
   255
3 RgbColor Color3(0,200,0);           // each color as passed in

```

- Q15.** `Shape` declares virtual `double area() const = 0;`. `Circle` inherits `Shape` but does not override `area()`. What is true?
- Q16.** What is the difference between an abstract class and an interface?. You should include in your discussion:
- Q17.** Discuss Early and Late binding. This is another topic we didn't touch on but is important. You should have these keywords present when answering this question: static, dynamic, virtual, abstract, interface. I can't say it enough, try to use examples in your discussion.
- Q18.** Which class has the highest degree of abstraction in a multilevel inheritance relationship of five levels?
- Q19.** Multiple Inheritance Workarounds (Interfaces) - Common patterns to avoid complexity.
- Mix-in classes or interface approach in languages like C++ and Java.

- Q20.** An abstract method in C++ is basically a regular method except it's declared as a pure virtual function (= 0). We sometimes call this a(n) __ in other OOP languages.
- Q21.** Discuss Early and Late binding. You should have these keywords present when answering this question: static, dynamic, virtual, abstract, interface. I can't say it enough, try to use examples in your discussion.
- Q22.** In the snippet below, what will line 20 (the last line) print to std out?

Polymorphism

- Q23.** (*Code*) There are 3 major concepts when we think about OOP. What are they? Use layman's terms to define each word, give examples if possible. Answer: PIE Polymorphism Inheritance Encapsulation
- Q24.** (*Code*) Polymorphism - Ability for methods to take many forms (e.g., method overriding, method overloading). - Dynamic vs. static binding.
- Q25.** If same message is passed to objects of several different classes and all of those can respond in a different way, what is this feature called?

- Q26.** Examples - Polymorphic containers (e.g., storing a list of **Shape**). - Polymorphic function calls based on the actual object type.
- Q27.** Explain what polymorphism is and give some examples.
- Q28.** What keyword must a method have to ensure that run time polymorphism can happen?
- Q29.** Which among the following is the main characteristic of class?
- Q30.** Use of pointers or reference to an abstract class gives rise to which among the following feature?
- Q31.** What is the one thing that is necessary for run time polymorphism?
- Q32.** Discuss Early and Late binding. Hint:

Virtual Functions

- Q33.** (*Code*) Inheritance Demo - Create **ElectricCar** derived from **Car** and override a method like **refuel** to do nothing or charge instead.
- Q34.** (*Code*) Short Answer. Explain and write an example in C++ describing the diamond problem and how to fix and/or avoid it. The diamond problem starts with a single

class that inherits from multiple classes that have the same base (super) class and then calling a method that exists in the base (super) class. For example below there is a print method in the Animal class that get inherited by two sub classes: If bat were to call Eat, which eat would it be calling? The copy in Mammal or the copy in WingedAnimal? It is ambiguous and won't work. The answer / fix is to use virtual inheritance so the system knows to only create a single instance of Animal for both sub classes when they are constructed. Virtual inheritance ensures there will only be one copy of the base (super) clas

```

1 //source: https://en.wikipedia.org/wiki/Virtual\_inheritance
2 struct Animal {
3     virtual ~Animal() = default;
4     virtual void Eat() {}
5 };
6
7 struct Mammal: Animal {
8     virtual void Breathe() {}
9 };
10
11 struct WingedAnimal: Animal {
12     virtual void Flap() {}
13 };
14
15 // A bat is a winged mammal
16 struct Bat: Mammal, WingedAnimal {};
17
18 Bat bat;

```

Q35. Widget::getCount() is called without creating any Widget object. This is valid because getCount() is declared:

Q36. Which is a correctly declared pure virtual function?

Q37. The primary purpose of the override keyword (C++11) is to:

Q38. The diamond problem in C++ is resolved using:

Q39. Which function best describe the concept of polymorphism in programming languages?

Q40. A virtual destructor is required when:

Constructors

Q41. (Code) Write a class definition and implementation to represent a Being. Your class should include the following data members: age, height, weight, classification (alien or terrestrial) and color (gray,green,orange,purple,etc.). Include two constructors and methods to set and get each data member.

```

1  class Being{
2      int age;
3      double height;
4      double weight;
5      string classification;
6      string color;
7  public:
8      Being(){
9          age = height = weight = 0;
10         classification = color = "";
11     }
12     Being(int a, double h, double w, string cls, string col): age
        {a}, height{h}, weight{w}, classification{cls}, color{col}
        {}
13     void setAge(int a){age = a;}
14     void setHeight(double h){height = h;}
15     void setWeight(double w){weight = w;}
16     void setClass(string c){classification = c;}
17     void setColor(string c){color = c;}
18
19     int getAge(){return age;}
20     double getHeight(){return height;}
21     double getWeight(){return weight;}
22     void getClass(){return classification;}
23     void getColor(){return color;}
24
25 };

```

Q42. (Code) Write a Point3D class definition that will represent a 3D point. Assume all values to be integers. Do not add any setters or getters. Include: Include a default constructor that sets each data member to zero. Include an overloaded constructor to init each data member. Include a copy constructor. Add a copy constructor.

```

1  [
2  frame=lines,
3  % bgcolor=LightGray,
4  linenos
5  ]
6  {c++}
7  class Point3D{
8      int x;
9      int y;
10     int z;
11 public:
12     Point3D(): x{0}, y{0}, z{0}{}    // default constructor using
        init lists
13     Point3D(){                        // default written old way
14         x = y = z = 0;
15     }
16
17     Point3D(int x , int y , int z): x{x}, y{y}, z{z}{}    //
        overloaded using init lists
18     Point3D(int _x , int _y , int _z){                //
        overloaded written old way
19         x = _x;
20         y = _y;
21         z = _z;
22     }
23
24     Point3D(const Point3D &rhs){
25         this->x = rhs.x;
26         this->y = rhs.y;
27         this->z = rhs.z;
28     }
29
30 };

```

Q43. What will be the output of the following C++ code? (Hint: Default protection in a

class?)

- Q44.** In the following C++ code how many times the string 'A's constructor called' will be printed?
- Q45.** If a class have default constructor defined in private access, and one parameter constructor in protected mode, how will it be possible to create instance of object?
- Q46.** Given the following code snippet, determine if it would need a copy constructor and explain your choice.
- Q47.** Which rule suggests that if a class requires a copy constructor, it likely also requires a destructor and an overloaded assignment operator?
- Q48.** What happens if a class with a pointer does not define a copy constructor and the default one is used?
- Q49.** Base Class Initialization - Order of constructor calls. - Virtual destructors in C++ for safe polymorphic deletion.
- Q50.** Design Patterns (Optional) - Basics of the most common patterns (Singleton, Factory, Observer, etc.). - Encourages best-practice OOP design strategies.

Destructors

- Q51.** (*Trace*) What is the exact output of this program?

```

1 #include <iostream>
2 using namespace std;
3
4 struct Base {
5     ~Base() { cout << "~Base" << endl; }

```

```

6 };
7 struct Derived : Base {
8     ~Derived() { cout << "~Derived" << endl; }
9 };
10
11 int main() {
12     Base* p = new Derived();
13     delete p;
14     return 0;
15 }

```

Output:

Q52. *(Code)* You have a base class Vehicle and two derived classes Car and Truck. Objects of Car and Truck will be stored as Vehicle* pointers and deleted through those pointers. Write a minimal Vehicle class with a virtual destructor that prints " Vehicle", and a Car class with a destructor that prints " Car". Then write a short main() that creates a Car on the heap as a Vehicle*, deletes it, and show the expected output.

Q53. *(Code)* What problem occurs if only a destructor is defined?

Q54. Why must a base class destructor be declared virtual if derived objects will be deleted through a base class pointer?

- Q55.** Destructors are implicitly noexcept in C++11 and later. What is the consequence if a destructor throws an exception during stack unwinding?
- Q56.** `delete a;` is called where `Animal* a = new Dog();`. The destructor `Animal()` is not virtual. What happens?
- Q57.** In the hierarchy ABC (A is root), a C object is created then goes out of scope. What is the correct order of calls?

Access Modifiers

- Q58.** *(Code)* Prism Class Write a Prism class definition and implementation that will represent either a cylindrical prism or rectangular prism shape depending on how many parameters are used to instantiate the object. Assume all input values to be integers. Your prism class should calculate the volume correctly for each prism type. You should have a public `Volume` method that calls the appropriate private `Volume` method depending on the shape the object was instantiated as. Example usage:

```

1 Prism A(2,4);    // r=2,h=4
2 Prism B(2,4,6);  // w=2,h=4,l=6
3 cout<<A.Volume()<<endl; // prints: 50.26
4 cout<<B.Volume()<<endl; // prints: 48.0

```

- Q59.** *(Code)* Write a Prism class definition and implementation that will represent either a cylindrical prism or rectangular prism shape depending on how many parameters are used to instantiate the object. Assume all input values to be integers. Your prism class should calculate the volume correctly for each prism type. You should have a public `Volume` method that calls the appropriate private `Volume` method depending on the shape the object was instantiated as. Example usage:

```

1 Prism A(2,4);    // r=2,h=4
2 Prism B(2,4,6);  // w=2,h=4,l=6

```

```

3 cout<<A.Volume()<<endl; // prints: 50.26
4 cout<<B.Volume()<<endl; // prints: 48.0

```

- Q60.** Like private members, protected members are inaccessible outside of the class. However, they can be accessed by derived classes and friend classes/functions.? [2]Class Wizard is the print method in Character
- Q61.** If two classes combine some private data members and provides public member functions to access and manipulate those data members. Where is abstraction used?
- Q62.** You want a method with behavior similar to a virtual method (it is meant to be overridden) except that it does not have a method body. It just has a method signature. What kind of method should you use?
- Q63.** If a class combine some private data members and provides public member functions to access and manipulate those data members. Where is abstraction used?

Templates

- Q64.** Templates / Generics - Parametric polymorphism for writing reusable code. - Separating algorithm logic from data types (in C++ or Java generics).
- Q65.** What is the purpose of the template keyword in C++?

Composition

Q66. *(Code)* Given the two class definitions below: You need to write a definition for a Car class using the above two classes. You need to extend one, and use the other as a data member. This question boils down to composition vs inheritance. Explain your reasoning after you write you Car definition (bare bones definition).

```
1 class Engine {} // The Engine class.  
2 class Automobile {} // Automobile class which is parent to Car  
   class.
```

Q67. *(Code)* What is the main difference between a class and an object? Answer: Use the corresponding letters for the words above to fill in the blanks below. A Class has a Definition , and Implementation whereas an Object resides in Memory and has State.

Q68. What is the main difference between a class and an object? Use the corresponding letters for the words above to fill in the blanks below. A Class has a , and whereas an Object resides in and has .

Q69. Which relationship is best modeled with composition rather than inheritance?

Q70. The design principle “favor composition over inheritance” is recommended because composition:

Q71. Why would you implement composition using an id instead of a reference?

Static Members

Q72. *(Code)* Write a member function for Thingy to allow access to thingyCount. You can write your method as if you were writing it inline if you wish.

```
1 static int getThingyCount() {  
2     return thingyCount;  
3 }
```

Q73. *(Code)* Write a class called NumObjects that counts the number of objects in existence. You should assume that each object will be created and destroyed before your program ends. So your count should be equal to the number of existing objects. You need the static keyword.

Q74. Static Members - Class-level methods/variables vs. instance-level. - Typical use cases and examples (utility methods, counters, etc.).

Q75. Compute a global average of all dice rolls among all teams using static members (sumAllRolls and totalRolls).

Q76. Global Average (Static Members): - `static int sumAllRolls` stores the sum of every roll from all teams. - `static int totalRolls` stores how many total dice rolls have been recorded across all teams. - Each time we call `addDiceRoll()`, we update these `static` members, allowing us to compute the global average via `Team::getGlobalAverage()`.

Q77. Which statement about static member functions is true?

Move Semantics

Q78. (*Code*) Write a class named `Buffer` that wraps a heap-allocated `int` array. The class should have: a constructor taking `int` size that allocates the array, a move constructor that steals the array pointer and sets the source's pointer to `nullptr`, a move assignment operator, and a destructor. Do not define a copy constructor or copy assignment operator (delete them). Show the complete class definition.

Q79. In a move assignment operator, what must you do before transferring resources from the source object?

Q80. Inside a move constructor body, does the parameter declared as `T&&` other behave as an lvalue or rvalue?

Q81. When a function returns a local object by value, the compiler can often eliminate the copy entirely. What is this optimization called?

Q82. What is the primary purpose of a move constructor?

- Q83.** Which declaration correctly defines an rvalue reference to int?
- Q84.** After an object has been moved from using `std::move`, what is the correct statement about its state?
- Q85.** Why can `std::unique_ptr` be stored in a `std::vector` even though it cannot be copied?

Type Casting

- Q86.** (*Code*) Write a function named `safePet` that takes an `Animal*` (where `Animal` has a virtual destructor and `Dog` and `Cat` are derived classes with a `speak()` method). The function should use `dynamic_cast` to determine if the animal is a `Dog`. If it is, print "Good dog!"; otherwise print "Not a dog.". Show the complete function.
- Q87.** Which cast should be used to convert a double to an int in C++?
- Q88.** What does `dynamic_cast` return when a pointer downcast fails at runtime?
- Q89.** What happens when `dynamic_cast` is used on a reference and the cast fails?
- Q90.** Which statement best describes `reinterpret_cast`?
- Q91.** What is the only cast that can remove the `const` qualifier from a pointer?

Q92. Which term describes casting a derived class pointer to a base class pointer?

Exceptions

Q93. (*Code*) Write a custom exception class named `InvalidAgeError` that inherits from `std::invalid_argument`. It should have a constructor taking an `int` `age` that passes an appropriate message (including the bad age value) to the parent constructor. Then write a function `setAge(int age)` that throws `InvalidAgeError` if `age` is negative or greater than 150, and otherwise prints the age. Show the exception class and the function.

Q94. What keyword is used to signal that an error condition has occurred and transfer control to a matching catch handler?

Q95. Why should exceptions be caught by reference (`catch(const MyException& e)`) rather than by value?

Q96. What is stack unwinding in the context of C++ exceptions?

Q97. What happens if a function declared `noexcept` throws an exception?

Q98. Which class is the root of the standard C++ exception hierarchy?

Q99. What does a bare throw statement (just `throw;` with no argument) do inside a catch block?

Smart Pointers

- Q100.** *(Code)* Write a function named `createBuffer` that takes an integer size and returns a `std::unique_ptr<int[]>` pointing to a heap-allocated array of that size with all elements initialized to 0. Do not use `new` directly — use `std::make_unique`.
- Q101.** Two objects A and B each hold a `shared_ptr` to the other. What will happen when both `shared_ptrs` go out of scope?
- Q102.** Smart Pointers (C++-Specific) - `std::unique_ptr`, `std::shared_ptr`, and `std::weak_ptr`.
- Automatic resource management.
- Q103.** What does the `use_count()` method on a `shared_ptr` return?
- Q104.** Why is `std::weak_ptr` used in conjunction with `shared_ptr`?
- Q105.** Why is `std::make_unique` preferred over calling `new` directly with `unique_ptr`?
- Q106.** What is the main difference between `shared_ptr` and `unique_ptr`?

RAII (Resource Acquisition Is Initialization)

- Q107.** *(Code)* Write a RAII class named `FileHandle` that wraps a C-style `FILE*` pointer. The constructor should take a `const char*` filename and a `const char*` mode, call `fopen` to open the file, and store the result. The destructor should call `fclose` if the file pointer

is not null. Include a method `isOpen()` that returns `bool`. Show the complete class definition.

- Q108.** Which problem does RAII solve that manual resource management (e.g., `explicit delete` or `close()`) does not handle reliably?
- Q109.** If an exception is thrown inside a scope where an RAII object was constructed, what happens to the RAII object?
- Q110.** What happens if resource acquisition fails inside an RAII constructor (e.g., the file cannot be opened)?
- Q111.** In RAII, when does the cleanup of a resource (such as closing a file or releasing memory) occur?
- Q112.** Why does RAII provide exception safety for resource management?
- Q113.** `std::lock_guard` and `std::mutex` is an example of RAII because:

Object Slicing

- Q114.** (*Code*) The following function causes object slicing. Rewrite it (and any related function calls) so that virtual dispatch works correctly for a hierarchy where `Animal` is the base class and `Dog` and `Cat` are derived classes, each with a virtual `speak()` method. The function should accept any `Animal` subtype and call `speak()` polymorphically. Show the

corrected function signature and a brief example of calling it. void makeNoise(Animal a) a.speak();

Q115. Which container type correctly stores a heterogeneous collection of polymorphic objects without slicing?

Q116. Why does defining a deleted or protected copy constructor in the base class help prevent accidental object slicing?

Q117. When a derived object is sliced into a base object, what happens to virtual dispatch?

Q118. In which situation does object slicing most commonly occur?

Q119. What is wrong with storing polymorphic objects in a `std::vector<Base>`?

Rule of Three / Five

Q120. (*Trace*) What is the exact output of this program, and what memory error (if any) occurs at the end?

```

1 #include <iostream>
2 using namespace std;
3
4 struct Arr {
5     int* data;
6     int size;
7     Arr(int n) : size(n), data(new int[n]) {
8         for (int i = 0; i < n; i++) data[i] = i;

```

```

9      }
10     ~Arr() { delete[] data; }
11     // No copy constructor defined
12 };
13
14 int main() {
15     Arr a(3);
16     Arr b = a;           // compiler-generated shallow copy
17     b.data[0] = 99;
18     cout << a.data[0] << endl; // what prints?
19     return 0;           // both destructors fire here
20 }

```

Output:

- Q121.** (Code) Write a class named StringWrapper that stores a heap-allocated C-string (char*). Implement the Rule of Three: a constructor taking const char*, a copy constructor that performs a deep copy, a copy assignment operator that handles self-assignment and performs a deep copy, and a destructor that frees the memory. Show the complete class definition.
- Q122.** A class manages a heap-allocated array with a raw pointer. If only a destructor is defined (but not a copy constructor), what bug occurs when the object is copied?
- Q123.** When a class defines a destructor but no copy constructor, what does the compiler generate for the copy constructor?
- Q124.** A class defines a copy constructor and copy assignment operator but no move constructor. What happens when an rvalue of that class is passed to a function expecting the class by value?

- Q125.** A class manages a heap-allocated resource. It defines a copy constructor and destructor, but no copy assignment operator. What problem arises from this?

Multiple Inheritance

- Q126.** (*Code*) Explain or write an example in C++ describing the diamond problem and how to fix/avoid it.

- Q127.** What is the diamond problem? This is a question about multiple inheritance and its potential problems. Use examples when possible, but explain thoroughly.

- Q128.** Single vs. Multiple Inheritance - Language differences (e.g., C++ vs. Java). - Diamond problem and virtual inheritance in C++.

- Q129.** Which type of inheritance leads to diamond problem?

- Q130.** Which programming language restricts the use of multiple inheritance?

Operator Overloading

- Q131.** (*Trace*) Rewrite the following struct and add an overload for ostream: The output should look like: First: firstNameValue Last: lastNameValue Grades: 1. 33 2. 88 3. 99 ... 10. 87 Answer:

```

1 struct Grades{
2     string fname;
3     string lname;

```

```

4         int grades[10];
5     };

```

Output:

Q132. (Code) Assignment operator Overload the assignment operator for our 3D point class. What must we check for when overloading the assignment operator and why? (a) (b)

```

1  [
2  frame=lines,
3  % bgcolor=LightGray,
4  linenos
5  ]
6  {c++}
7
8  friend ostream& operator<<(ostream &os, const Point3D &rhs){
9      return os << "[" << rhs.x << ", "<< rhs.y << ", "<< rhs.z << "] ";
10 }

```

Q133. (Code) Overload ostream for the RgbColor class so it prints the color values with the following format: [r,g,b] (replacing r,g,b with the objects numeric values). Assume you are defining this within the class definition, so no scope resolution operator necessary.

```

1  friend ostream& operator<<(ostream& os, const RgbColor& rhs){
2      os<< "[" << rhs.r << ", "<< rhs.g << ", "<< rhs.b << "] ";
3      return os;
4  }

```

- Q134.** There is a set of operators that are considered destructive. Which operators are they, and what does this mean?
- Q135.** If a class requires a user-defined copy constructor it probably needs a user-defined , and a user-defined as well.

Memory Management

- Q136.** (Code) Given the following class definition, write a destructor for the class to delete the dynamically allocated memory.

```

1  class arrayClass{
2      int *A;
3      int size;
4      arrayClass(){
5          size = 10;
6          A = new int[size];
7      }
8      arrayClass(int s){
9          size = s;
10         A = new int[size];
11     }
12     ...
13     // destructor needed define - below outside of class
14 };

```

- Q137.** Online Tutorials & Practice - Codecademy, Udemy, Coursera courses on OOP. - Community resources like Stack Overflow for Q&A.
- Q138.** What will be the output of the program given below?

Q139. When discussing classes and objects, we can say that:

Q140. Which code creates a new object from the Employee class?

Function Overloading

Q141. (*Code*) What possibilities are there for overloading the delContact method? Could you use their phone number? What about the id? There is one good answer, so choose wisely. 1mm

Q142. (*Code*) Overriding vs. Overloading - Method overloading (same name, different parameters). - Method overriding (subclass redefines a parent class method).

Q143. Overload ostream for our 3D class so it prints the values like so: where ,, and would be integers (obviously).

Q144. The principle of allows us to create new classes based on existing classes.

Friend Functions

- Q145.** What will be the output of the following C++ code?
- Q146.** Which is false for a member function of a class?
- Q147.** What does the friend keyword in C++ allow us to do?

Encapsulation

- Q148.** (*Code*) On your answer sheet, write A-J and label each with abstraction or encapsulation: Abstraction shows only useful data by providing the most necessary details. Encapsulation hides internal working. Encapsulation solves problem at implementation level. Encapsulation wraps code and data together. Abstraction is focused mainly on what should be done. Encapsulation is focused on how it should be done. Encapsulation helps developers to organize code easily. Abstraction hides complexity. Abstraction solves problem at design level. Encapsulation hides the irrelevant details found in the code. <https://github.com/rugbyprof/2143-Object-Oriented-Programming/blob/master/Resources/00-OOP/05-AbsVSEnc.md> 05-AbsVSEnc.md
- Q149.** General Concepts: - Data Member - Method (Member Function) - Object - Instance - Encapsulation - Resource Management
- Q150.** Consider the following code and select the correct option.